



Presenter Guide — Session 1: Hello, Mobile World

Duration: 90 minutes · **Audience:** absolute beginners (no prior coding) · **Outcome:** every student runs a styled "About Me" card on their own phone.

At a glance

Block	Time	Topic
1	0:00–0:10	Welcome · goal · how it works · setup check
2	0:10–0:17	What is mobile dev · native vs cross-platform · why RN+Expo · mental model
3	0:17–0:24	Run an app on the phone · hot reload (the "aha")
4	0:24–0:35	JavaScript crash course (variables, types, arrays, objects, functions)
5	0:35–0:45	First component · JSX · core components (View/Text/Image)
6	0:45–0:55	Styling · Flexbox · live-code the Profile Card
7	0:55–1:20	Activity 1: About Me card (guided)
8	1:20–1:30	Recap · take-home · Session 2 teaser



If you're running late: the JS crash course (Block 4) is the most compressible — cut arrays/objects detail and let them meet those ideas live in the code. Never cut Block 3 (getting it on the phone) — that's the emotional hook.

Before class (presenter pre-flight)

- ☐ Projector mirrors **both** your editor (`App.js`) and, ideally, your phone screen.
- ☐ You can run `git checkout session-1 && npm install && npx expo start` and it loads on your phone.
- ☐ Have `snack.expo.dev` open in a browser tab as the fallback for students whose setup failed.

- ☐ Wi-Fi works for the room; know the `--tunnel` fallback.
 - ☐ Slides: `pdf/session-1-presenter.pdf` . Live-code target: the code in this guide / `session-1` branch `App.js` .
-

Block 1 — Welcome (0:00–0:10)

Talking points

- "By the end of today, an app **you** built is running on **your** phone." Repeat it; it's the promise.
- Show of hands: *who has never written a line of code?* Reassure them — they're the target audience.
- The 3-week arc, one sentence each: **Week 1** a screen → **Week 2** it reacts to you → **Week 3** it remembers your data.
- **Setup check:** ask everyone to run `node --version` . Anyone failing → pair them up or move to Snack so nobody is blocked.



ELABORATE: Set the emotional tone. Beginners are nervous. Normalize errors: "We will all see red error messages today — that's normal, not failure. I'll show you how to read them."

Block 2 — What is mobile development? (0:10–0:17)

Talking points

- **Native vs cross-platform:** historically you built an app *twice* (Kotlin for Android, Swift for iPhone). React Native = **write once in JavaScript, run on both**.
- **React Native vs Expo** — keep this distinction crisp:
 - *React Native* = the framework (the UI building blocks).
 - *Expo* = a friendly layer on top that removes native setup, gives us **Expo Go**, and bundles libraries (like the **SQLite** we use in Week 3).
- Drop real names: **Instagram, Discord, Shopify, Coinbase, Tesla** use React Native. This is real, not a toy.
- **Mental-model slide** is the anchor: walk *down* the arrows once. "You only ever touch the top two layers."



ELABORATE: Don't explain the "bridge" / native modules / architecture. For beginners it's pure distraction. Keep everything at the "one codebase → both phones" altitude.

Block 3 — Get it on the phone (0:17–0:24)

Do this live and have them follow.

1. Show `npx create-expo-app@latest my-app --template blank` (how to start from zero — they'll need it for the final project).
2. In class we use the repo: `git checkout session-1` → `npm install` → `npx expo start`.
3. **Scan the QR** — Android via Expo Go's scanner; iPhone via the Camera app.
4. **Hot reload demo:** change the text to your name, save, watch the phone update. **Pause here** until heads are up across the room.



ELABORATE: This is the single most important moment of Session 1. If a student's phone shows the app, they're hooked. Spend the time to get *everyone* there.

Troubleshooting: same Wi-Fi, disable VPN, `--tunnel` fallback, press `r` to reload, `npx expo start -c` to clear cache.

Block 4 — JavaScript crash course (0:24–0:35)

Teach **only** these, fast, and promise each reappears in real code within minutes:

- **Variables:** `const` (default) vs `let` (changes). Box-with-a-label analogy. Don't mention `var`.
- **Types:** string (quoted), number, boolean (`true` / `false`).
- **Arrays:** `["a", "b"]`, indexing from `0`, `.length`, `.push()`. → "a to-do list is just an array."
- **Objects:** `{ title: "Buy milk", done: false }`, dot access. → "a to-do item is an object."
- **Functions:** inputs → output; show the arrow-function form because React uses it everywhere.



ELABORATE: The highest-value idea here is **const vs let**. Spend the most time there. For arrays+objects, your goal is just recognition, not mastery — they'll *use* them next week.

Resist tangents (no `map` / `reduce` / `this` /closures today).

Block 5 — First component & core building blocks (0:35–0:45)

Talking points

- A **component is a function that returns UI**. Walk the 4 parts of `App.js` (import, function, return JSX, export).
- **JSX rules**: one parent wrapper · `{ }` to inject JS · self-closing tags (`<Image />`).
- **Core components**:
 - `<View>` = a box; nest boxes to build layouts.
 - `<Text>` = **all** text must live here. Teach the error now: *"Text strings must be rendered within a component."*
 - `<Image>` = `uri` for internet images (needs width+height) or `require()` for local.

 **ELABORATE:** The `<Text>` rule is the #1 crash for beginners. Say it twice, show the error on purpose if you can, so it's familiar instead of scary later.

Block 6 — Styling + live-code the card (0:45–0:55)

Talking points

- Styles are **JS objects**: `camelCase` keys, numbers (no `"px"`), strings for colors. `StyleSheet.create` keeps them tidy.
- **Flexbox**: default direction is `column` in RN (web is `row` !). `flexDirection: 'row'` for side-by-side.
- Centering recipe: `flex:1 + justifyContent: 'center' + alignItems: 'center'` .
- `padding` (inside) vs `margin` (outside); `borderRadius` for rounded corners.

Live-code the Profile Card (the exact target students will personalize). Build it incrementally:

View → Image → name/role Text → bio box → info rows. Narrate every line. Don't expect keystroke-for-keystroke following yet — they watch now, build in the activity.

✓ Full live-code / solution code (matches **session-1** branch **App.js**)

```
import { StatusBar } from 'expo-status-bar';
import { Image, ScrollView, StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <ScrollView style={styles.screen} contentContainerStyle={styles.content}>
      <StatusBar style="light" />

      <View style={styles.card}>
        <Image
          style={styles.avatar}
          source={{ uri: 'https://i.pravatar.cc/300?img=12' }}
        />

        <Text style={styles.name}>Juan dela Cruz</Text>
        <Text style={styles.role}>Future Mobile Developer</Text>

        <View style={styles.bioBox}>
          <Text style={styles.bioText}>
            🙋 Hi! I'm learning to build mobile apps with React Native & Expo.
            This entire screen is my very first app – made on my own phone!
          </Text>
        </View>

        <View style={styles.infoRow}>
          <Text style={styles.infoLabel}>📍 Location</Text>
          <Text style={styles.infoValue}>Batangas City, PH</Text>
        </View>
        <View style={styles.infoRow}>
          <Text style={styles.infoLabel}>🎓 School</Text>
          <Text style={styles.infoValue}>University of Batangas</Text>
        </View>
        <View style={styles.infoRow}>
          <Text style={styles.infoLabel}>💡 Learning</Text>
          <Text style={styles.infoValue}>React Native + Expo</Text>
        </View>
      </View>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: '#4338CA' },
  content: { padding: 20, paddingTop: 80, alignItems: 'center' },
  card: {
    width: '100%', backgroundColor: '#FFFFFF', borderRadius: 24, padding: 24,
    alignItems: 'center', elevation: 6, shadowColor: '#000', shadowOpacity: 0.15,
```

```

    shadowRadius: 12, shadowOffset: { width: 0, height: 6 },
  },
  avatar: {
    width: 120, height: 120, borderRadius: 60,
    borderWidth: 4, borderColor: '#06B6D4', marginBottom: 12,
  },
  name: { fontSize: 26, fontWeight: 'bold', color: '#0F172A' },
  role: { fontSize: 16, color: '#06B6D4', fontWeight: '600', marginBottom: 16 },
  bioBox: { backgroundColor: '#EEF2FF', borderRadius: 16, padding: 16, marginBottom: 20 },
  bioText: { fontSize: 15, color: '#475569', lineHeight: 22, textAlign: 'center' },
  infoRow: {
    flexDirection: 'row', justifyContent: 'space-between', width: '100%',
    paddingVertical: 12, borderTopWidth: 1, borderTopColor: '#E2E8F0',
  },
  infoLabel: { fontSize: 15, color: '#64748B', fontWeight: '600' },
  infoValue: { fontSize: 15, color: '#0F172A', fontWeight: '600' },
});

```

💡 The round avatar trick: `borderRadius` = **half** the width/height (120 → 60).

Block 7 — Activity 1 (0:55–1:20)

Hand off to `activities/session-1-activity.md`. Students personalize the card (their photo, name, bio, 3 info rows). Circulate constantly.

Common student errors & fixes

Symptom	Cause	Fix
● "Text strings must be rendered within a <code><Text></code> "	Raw text loose in a <code><View></code>	Wrap it in <code><Text>...</Text></code>
Image doesn't appear	No <code>width</code> / <code>height</code> , or bad URL	Add <code>style={{ width, height }}</code> ; check the <code>uri</code>
Everything crammed top-left	Forgot Flexbox alignment	Add <code>alignItems: 'center'</code> / <code>padding</code> on the parent
Avatar is a square, not a circle	<code>borderRadius</code> too small	Set it to half the size (120 → 60)
Red screen, "Unexpected token"	Missing <code>}</code> / <code>></code> / unclosed tag	Read the line number; check the tag/brace just above
Styles "do nothing"	Used <code>class=</code> or kebab-case	It's <code>style={styles.x}</code> and <code>camelCase</code> keys
App won't load on phone	Different Wi-Fi / VPN	Same network; <code>npx expo start --tunnel</code>



Facilitation: Resist fixing their keyboard for them. Ask "what's the red text say? what line?" — teach error-reading. Celebrate personalized cards out loud; it builds momentum.

Block 8 — Wrap-up (1:20–1:30)

- **Active recall** (ask the room): "What wraps all text?" (`<Text>`) · "Default flex direction?" (`column`) · "const or let for a value that changes?" (`let`).
- **Take-home:** restyle the card + add one new section (skills/hobbies/quote).
- **Tease Session 2:** "Your card is *frozen* — it can't do anything. Next week it reacts to taps and typing, and we build a To-Do list that grows." Remind them: laptop + charged phone.
- End **on time**.

Answer key — take-home

Any reasonable personalization is full credit. Look for: a new `<View>` section with its own styled `<Text>` children, correct `<Text>` wrapping, and at least one new style property used (color/background/borderRadius). The point is *confidence and ownership*, not correctness.